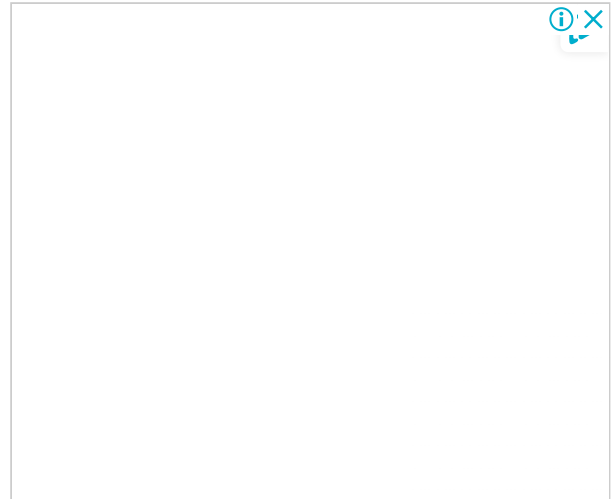


Activation Functions: Sigmoid vs Tanh

FEATURED VIDEOS



(<https://ads.freestar.com/?>)

Last updated: February 13, 2025



Written by: Panagiotis Antoniadis

(<https://www.baeldung.com/cs/author/panagiotisantoniadis>)



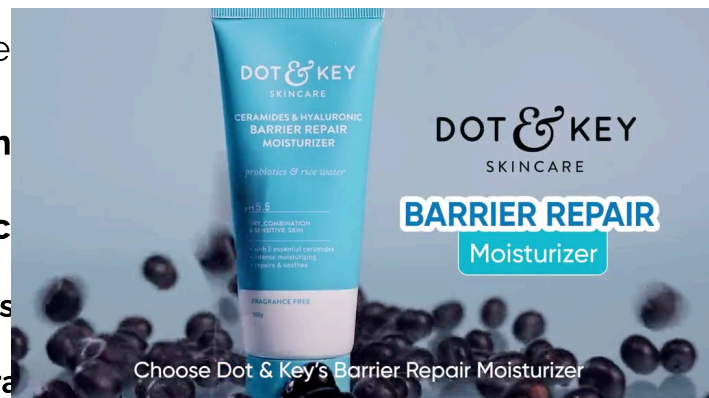
Reviewed by: Michal Aibin (<https://www.baeldung.com/cs/author/michal-aibin>)

Deep Learning (<https://www.baeldung.com/cs/tag/deep-learning>)

Machine Learning (<https://www.baeldung.com/cs/tag/machine-learning>)

Neural Networks (<https://www.baeldung.com/cs/tag/neural-networks>)

Training (<https://www.baeldung.com/cs/tag/training>)



In this tutorial, **we'll talk about the sigmoid and the tanh activation functions**. First, we'll briefly introduce activation functions, then present these two important functions, compare them and provide a detailed example.

Finally, we'll provide the implementation details of the sigmoid and the tanh activation functions in Python.

2. Activation Functions

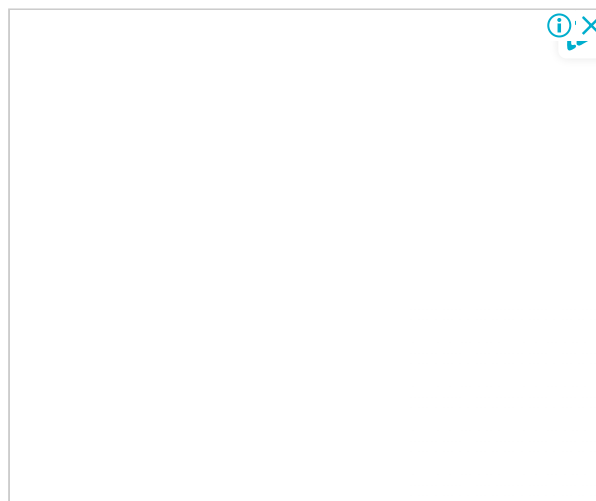
An essential building block of a neural network

(https://en.wikipedia.org/wiki/Artificial_neural_network) is the activation function that decides whether a neuron will be activated or not. Specifically, the value of a neuron in a feedforward neural network is calculated as follows:

$$Y = \sum_{i=1}^m (x_i * w_i) + b$$

where x_i are the input features, w_i are the weights, and b is the bias of the neuron.

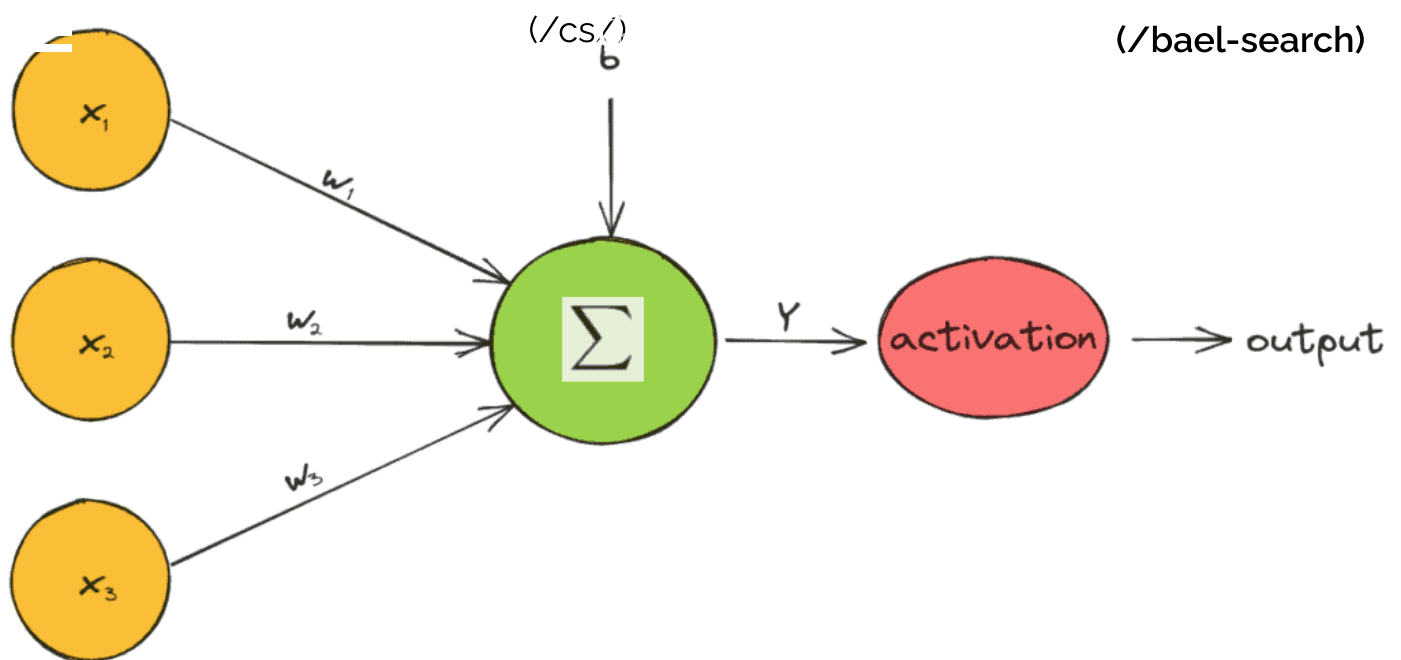
Then, an activation function f is applied at the value of every neuron and decides whether the neuron is active or not:



(https://ads.freestar.com/?utm_campaign=branding&utm_medium=banner&utm_source=baeldung.com&utm_content=baeldung-ad-aderboard_mid_1)

$$\text{output} = f(Y)$$

In the figure below, we can see diagrammatically how an activation function works:

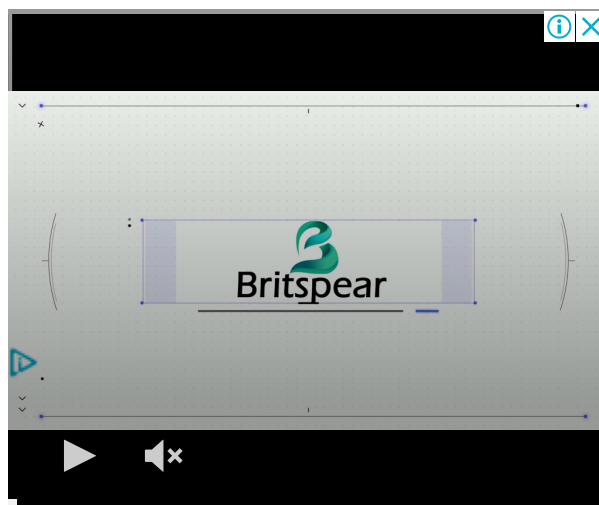


Thus, the activation functions are univariate and non-linear since a network with a linear activation function is equivalent to just a linear regression model (/cs/linear-vs-logistic-regression). Due to the non-linearity of activation functions, neural networks can capture complex semantic structures and achieve high performance.

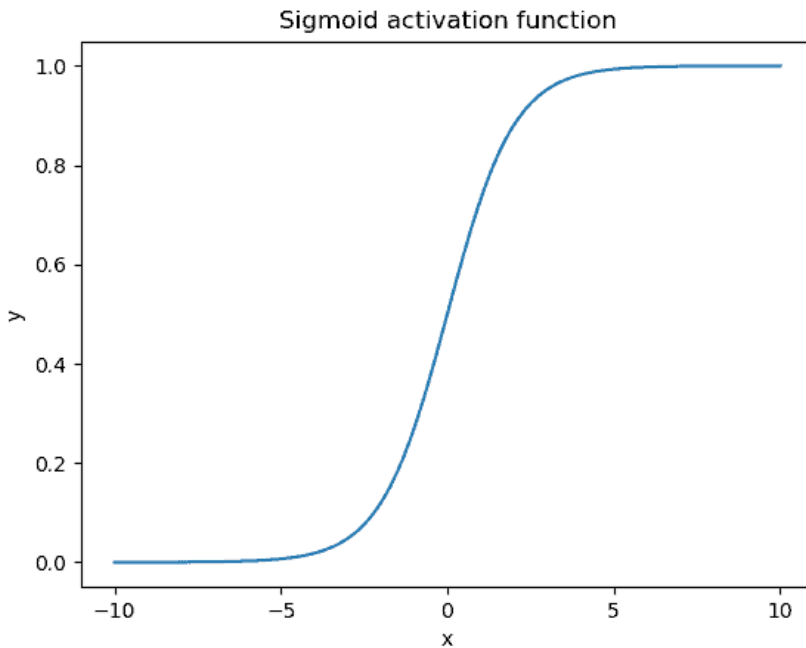
3. Sigmoid

The sigmoid (https://en.wikipedia.org/wiki/Sigmoid_function) activation function (also called logistic function) takes any real value as input and outputs a value in the range (0, 1). It is calculated as follows:

$$s(x) = \frac{1}{1+e^{-x}}$$



Here, x is the output value of the neuron. Moreover, we can see the plot of the sigmoid function when the input lies in the range $[-10, 10]$:



As expected, the sigmoid function is non-linear and bounds the value of a neuron in the small range of $(0, 1)$. **When the output value is close to 1, the neuron is active and enables the flow of information, while a value close to 0 corresponds to an inactive neuron.**

Also, an important characteristic of the sigmoid function is that **it tends to push the input values to either end of the curve (0 or 1) due to its S-like shape**. In the region close to zero, if we slightly change the input value, the respective changes in the output are very large and vice versa. For inputs less than -5, the output of the function is almost zero, while for inputs greater than 5, the output is almost one.

Finally, **the output of the sigmoid activation function can be interpreted as a probability** since it lies in the range $[0, 1]$. That's why it is also used in the output neurons of a prediction task.

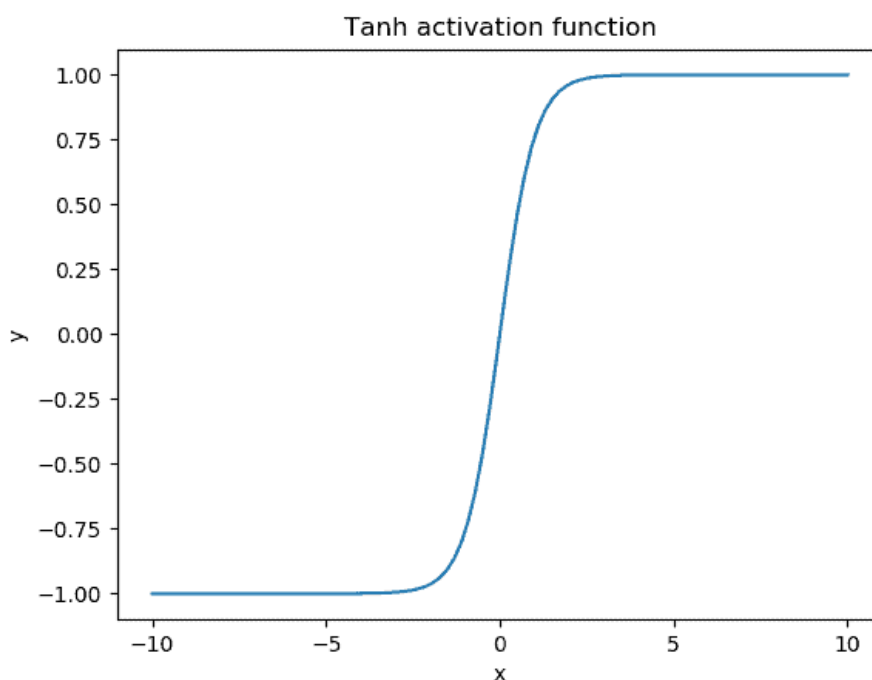
4. Tanh



Another activation function that is common in deep learning is the tangent hyperbolic (https://en.wikipedia.org/wiki/Hyperbolic_functions) function simply referred to as the tanh function. It is calculated as follows:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}} = \frac{2 - (1 + e^{-2x})}{1 + e^{-2x}} = \frac{2}{1 + e^{-2x}} - 1 = 2s(2x) - 1$$

We observe that the **tanh function is a shifted and stretched version of the sigmoid**. Below, we can see its plot when the input is in the range $[-10, 10]$:



The output range of the tanh function is $(-1, 1)$ and presents a similar behavior with the sigmoid function. Thus, the main difference is the fact that **the tanh function pushes the input values to 1 and -1 instead of 1 and 0**.

5. Comparison

Both activation functions have been extensively used in neural networks since they can learn complex structures. Now, let's compare them, presenting their similarities and differences. ^(/cs/) ^(/bael-search)

5.1. Similarities (<https://drafts.baeldung.com/cs/wp-admin/post.php?post=127901&action=edit>)

As we mentioned earlier, the tanh function is a stretched and shifted version of the sigmoid. Therefore, there are a lot of similarities.

(<https://drafts.baeldung.com/cs/wp-admin/post.php?post=127901&action=edit>)

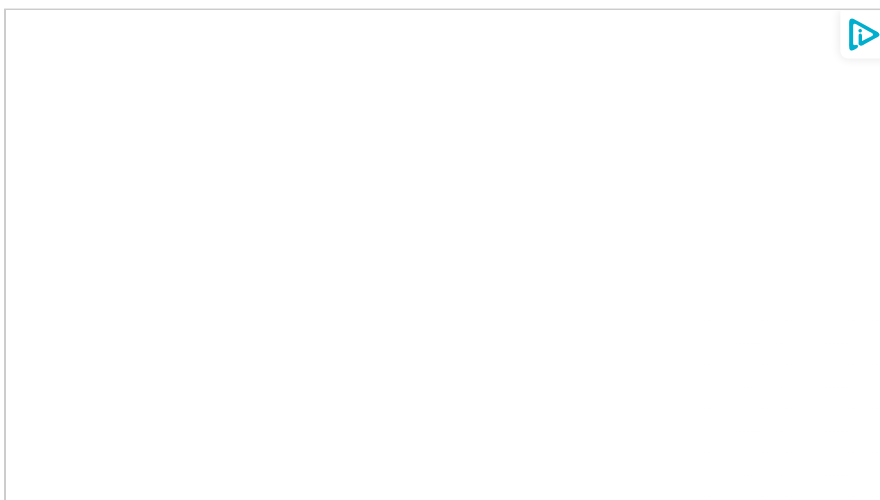
Both functions belong to the S-like functions that suppress the input value to a bounded range. This helps the network to keep its weights bounded and prevents the exploding gradient problem where the value of the gradients becomes very large. (<https://drafts.baeldung.com/cs/wp-admin/post.php?post=127901&action=edit>)

5.2. Differences (<https://drafts.baeldung.com/cs/wp-admin/post.php?post=127901&action=edit>)

An important difference between the two functions is the behavior of their gradient. Let's compute the gradient of each activation function:

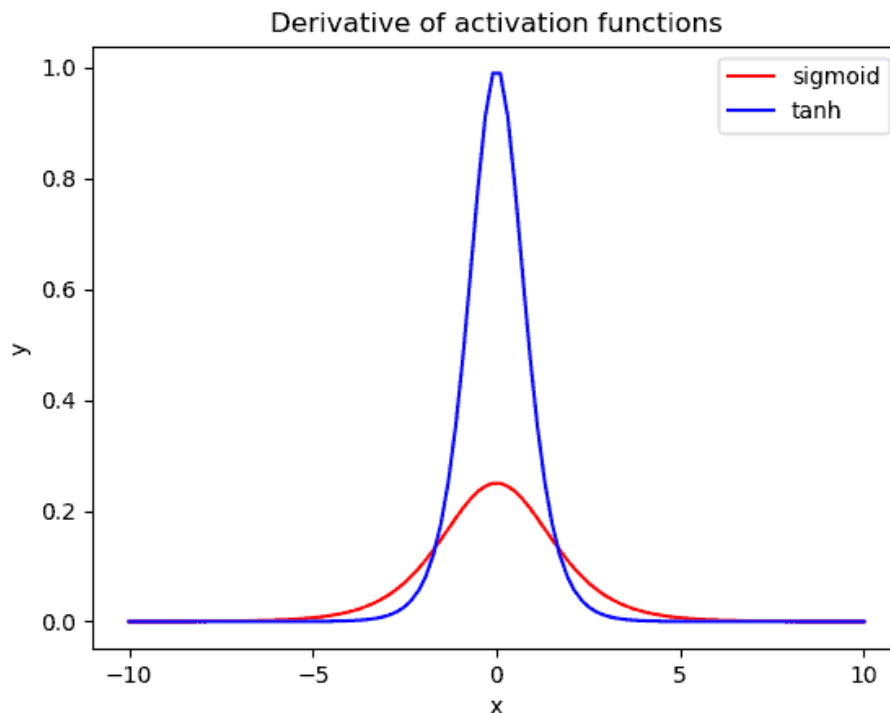
(<https://drafts.baeldung.com/cs/wp-admin/post.php?post=127901&action=edit>)

$s'(x) = s(x)(1 - s(x))$ (<https://drafts.baeldung.com/cs/wp-admin/post.php?post=127901&action=edit>)



$\tanh'(x) = 1 - \tanh^2(x)$ (<https://drafts.baeldung.com/cs/wp-admin/post.php?post=127901&action=edit>)

Below, we plot the gradient of the sigmoid (red) and the tanh (blue) activation function: (<https://drafts.baeidung.com/cs/wp-admin/post.php?post=127901&action=edit>) ([/cs/](#)) ([/bael-search](#))



When using these activation functions in a neural network, our data are usually centered around zero. Hence, we should focus on the behavior of each gradient in the region near zero.

Additionally, we observe that the gradient of tanh is four times greater than the gradient of the sigmoid function. This means that using the tanh activation function results in higher gradient values during training and higher updates in the weights of the network. **So, if we want strong gradients and big learning steps, we should use the tanh activation function.**

Another difference is that the output of tanh is symmetric around zero, leading to faster convergence.



6. Vanishing Gradient

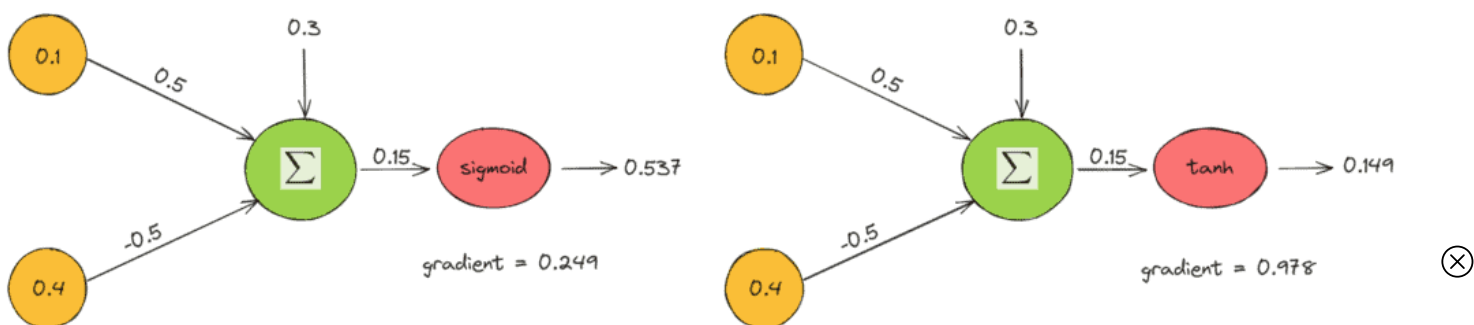
Despite their benefits, both functions present the so-called vanishing gradient problem (https://en.wikipedia.org/wiki/Vanishing_gradient_problem).

In neural networks, the error is backpropagated through the hidden layers of the network and updates the weights. **In case we have a very deep neural network and bounded activation functions like the ones above, the amount of error decreases dramatically after it is backpropagated through each hidden layer.** Hence, the error is almost zero at the early layers, and the weights of these layers are not

updated properly. Thus, the ReLU ([https://en.wikipedia.org/wiki/Rectifier_\(neural_networks\)](https://en.wikipedia.org/wiki/Rectifier_(neural_networks))) activation function can fix the vanishing gradient problem. ([bael-search](#))

7. Example

Finally, we'll present an example of applying these activation functions in a simple neuron of two input features $x_1 = 0.1, x_2 = 0.4$ and weights $w_1 = 0.5, w_2 = -0.5$. Therefore, we can see the output value and the gradient when we use the sigmoid (left) and the tanh (right) activation function:



The above example verifies our previous comments. Furthermore, **the output value of tanh is closer to zero, and the gradient is four times greater.**

8. Implementation in Python

Let's discuss the implementation details of the sigmoid and the tanh activation functions in Python.

8.1. Implementation of Sigmoid Function

(/es/)

(/bael-search)

We'll explore three ways to implement the sigmoid activation function in Python.

Our first approach is to use the *math*

(<https://docs.python.org/3/library/math.html>) module. At the start of the implementation, we define a function for the implementation. Moreover, we add an instruction to implement the sigmoid activation function. Finally, the *sigm* function returns the value *s* calculated using the sigmoid activation function:

```
import math
def sigm(x):
    s = 1 / (1 + math.exp(-x))
    return s
```



Alternatively, **we can implement the sigmoid activation function using the *numpy*** (<https://numpy.org/>) **library.** Let's take a look at the implementation:

```
import numpy as np
def sigm(x):
    k = np.exp(-x)
    s = 1 / (1 + k)
    return s
```



Finally, **we can use the *SciPy*** (<https://scipy.org/>) **library in Python for the implementation.** Here, we're using the *expit* function from the *SciPy* library:

```
from scipy.special import expit
def sigm(x):
    s = expit(x)
    return s
```



Now, let's provide some input values:



```
ip = [-4, -3, -2, 0, 2, 3, 4]
for i in ip:
    op = sigm(i)
    print(f"Input: {i}, Output: {op}")
```



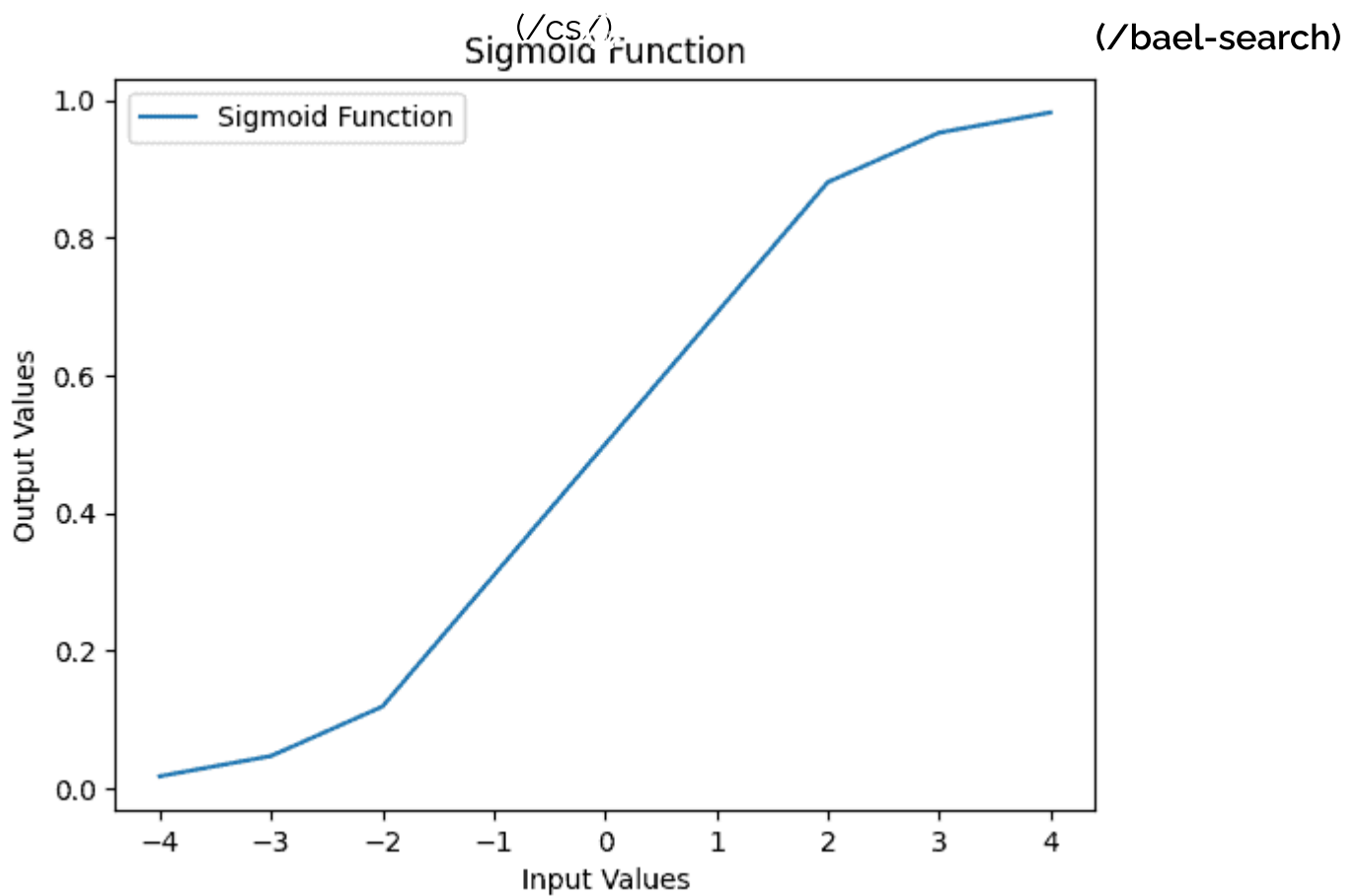
Let's take a look at the output sigmoid values:

```
Input: -4, Output: 0.01798620996209156
Input: -3, Output: 0.04742587317756678
Input: -2, Output: 0.11920292202211755
Input: 0, Output: 0.5
Input: 2, Output: 0.8807970779778823
Input: 3, Output: 0.9525741268224334
Input: 4, Output: 0.9820137900379085
```

Finally, we can plot the values using the *matplotlib* (<https://matplotlib.org/>) library in Python:

```
import matplotlib.pyplot as plt
ip = [-4, -3, -2, 0, 2, 3, 4]
op = [sigm(i) for i in ip]
plt.plot(ip, op, label='Sigmoid Function')
plt.xlabel('Input Values')
plt.ylabel('Output Values')
plt.title('Sigmoid Function')
plt.legend()
plt.show()
```

Let's take a look at the output figure:



8.2. Implementation of Tanh Function

Similar to the implementation of the sigmoid activation function, we'll explore three ways to implement the tanh activation function.

First, we use the *math* library:

```
import math
def tanh(x):
    t = (math.exp(x) - math.exp(-x)) / (math.exp(x) + math.exp(-x))
    return t
```

Next, we utilize the *tanh* function from the *numpy* library to calculate the calculate the hyperbolic tangent of an input value:

```
import numpy as np
def tanh(x):
    t = np.tanh(x)
    return t
```

Alternatively, we can also use the *tanh* function from the *SciPy* library to implement the tanh activation function:

```
from scipy.special import tanh
def tanh(x):
    t = tanh(x)
    return t
```

(/bael-search)

Now, we provide some input values:

```
ip1 = [-4, -3, -2, 0, 2, 3, 4]
for j in ip1:
    op1 = sigm(j)
    print(f"Input: {j}, Output: {op1}")
```

Let's take a look at the output values:

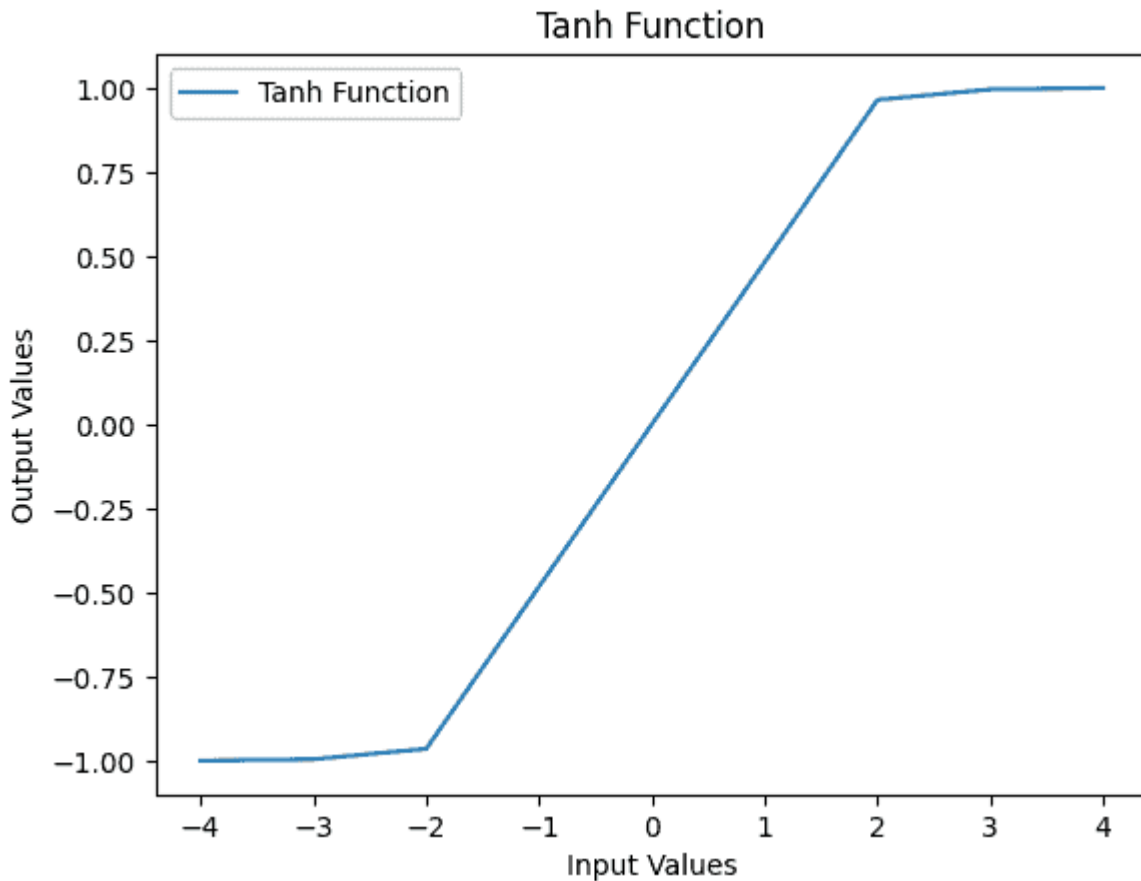
```
Input: -4, Output: -0.9993292997390669
Input: -3, Output: -0.9950547536867306
Input: -2, Output: -0.964027580075817
Input: 0, Output: 0.0
Input: 2, Output: 0.964027580075817
Input: 3, Output: 0.9950547536867306
Input: 4, Output: 0.9993292997390669
```

Furthermore, we utilize the *matplotlib* library for visualization:

ch)

```
import matplotlib.pyplot as plt
```

Finally, let's take a look at the figure that plots all the output values:



9. Conclusion

In this tutorial, we talked about two activation functions, the tanh and the sigmoid.

We explored the basic idea and compared the two functions with an example.

Finally, we provided the implementation details of the tanh and the sigmoid activation functions in Python.



CATEGORIES

[ALGORITHMS \(/CS/CATEGORY/ALGORITHMS\)](#)
[ARTIFICIAL INTELLIGENCE \(/CS/CATEGORY/AI\)](#)
[CORE CONCEPTS \(/CS/CATEGORY/CORE-CONCEPTS\)](#)
[DATA STRUCTURES \(/CS/CATEGORY/DATA-STRUCTURES\)](#)
[LATEX \(/CS/CATEGORY/LATEX\)](#)
[NETWORKING \(/CS/CATEGORY/NETWORKING\)](#)
[SECURITY \(/CS/CATEGORY/SECURITY\)](#)

SERIES

[GRAPHS TUTORIAL \(HTTPS://WWW.BAELDUNG.COM/CS/GRAPHS-SERIES\)](https://www.baeldung.com/cs/graphs-series)
[NEURAL NETWORKS SERIES \(HTTPS://WWW.BAELDUNG.COM/CS/NEURAL-NETWORKS-SERIES\)](https://www.baeldung.com/cs/neural-networks-series)
[LATEX SERIES \(HTTPS://WWW.BAELDUNG.COM/CS/LATEX-SERIES\)](https://www.baeldung.com/cs/latex-series)



ABOUT

[ABOUT BAELDUNG \(HTTPS://WWW.BAELDUNG.COM/ABOUT\)](https://www.baeldung.com/about)
[BAELDUNG ALL ACCESS \(/COURSES\)](#)
[THE FULL ARCHIVE \(/CS/FULL_ARCHIVE\)](#)
[EDITORS \(HTTPS://WWW.BAELDUNG.COM/EDITORS\)](https://www.baeldung.com/editors)
[OUR PARTNERS \(HTTPS://WWW.BAELDUNG.COM/PARTNERS/\)](https://www.baeldung.com/partners/)
[PARTNER WITH BAELDUNG \(HTTPS://WWW.BAELDUNG.COM/PARTNERS/WORK-WITH-US\)](https://www.baeldung.com/partners/work-with-us)

EBOOKS (HTTPS://WWW.BAELDUNG.COM/LIBRARY/)
FAQ (HTTPS://WWW.BAELDUNG.COM/LIBRARY/FAQ)
BAELDUNG PRO (/MEMBERS/)

(/cs/)

(/bael-search)

TERMS OF SERVICE (HTTPS://WWW.BAELDUNG.COM/TERMS-OF-SERVICE)
PRIVACY POLICY (HTTPS://WWW.BAELDUNG.COM/PRIVACY-POLICY)
COMPANY INFO (HTTPS://WWW.BAELDUNG.COM/BAELDUNG-COMPANY-INFO)
CONTACT (/CONTACT)

PRIVACY MANAGER

